

ENUM- Enumerations (Enumerated type)

- enum Direction{ // this is an enum class.
North, // these are the constants.
South,
East,
West, }

- Enum** is a pre-defined set of **constants**.
- Enum is nothing but a special type of class.
- Internally => **enum Direction** → **class Direction**.
- Every enum class **@Override** the parent Enum class which is in java.lang package, internally.
- Example: enum Direction extends Enum <Directions> { }
- Every **constant** in enum is the object of the **enum name**.
- Each **Constants** in enum are internally by-default **static & final**.
- Default Type of each **constants** are the **enum Direction**.
- Therefore, internally it is like this.--> (**north constant**) of direction class is internally stored like this below, and same for South, East, West.
public static final Direction North = new Direction(); // ye object type ha to yhi inke object bn gye ha.
- Internally 'enum Direction' ka apna private constructor hota ha taki koi bhar se isaka object na bna ske.

```
public static void main(String[] args) {  
    Direction d1 = Direction.NORTH;  
    System.out.println(d1.getDegree());  
    Direction d2 = Direction.SOUTH;  
    System.out.println(d2.getDegree());  
}  
}  
enum Direction{ // this "Direction" is also a class.  
    NORTH(0), // these all constants are the objects of the "Direction Class".  
    SOUTH(180),  
    EAST(90),  
    WEST(270);  
  
    private int degree; // this should be 'private' and call through getters.  
  
    Direction(int degree){ // constructor of enum is always private, to hum sirf isko enum ke ander  
hi edit kr skte ha.  
        this.degree = degree;  
    }  
    public int getDegree(){ // getter to call the 'int degree'.  
        return this.degree;  
    }  
}
```

```
enum Direction {

    NORTH(0),
    EAST(90),
    SOUTH(180),
    WEST(270);

    private int degrees;

    Direction(int degrees) {
        this.degrees = degrees;
    }

    public int getDegrees() {
        return degrees;
    }
}
```

```
final class Direction extends Enum<Direction> {

    public static final Direction NORTH = new Direction(0);
    public static final Direction EAST = new Direction(90);
    public static final Direction SOUTH = new Direction(180);
    public static final Direction WEST = new Direction(270);

    private int degrees;

    private Direction(int degrees) {
        this.degrees = degrees;
    }

    public int getDegrees() {
        return degrees;
    }
}
```

Enum Direction Moves

```
public class EnumDirectionMove {
    public static void main(String[] args) {
        Direction d = Direction.NORTH;
        d.move();
    }
}

enum Direction {
    NORTH{ // Anonymus class--> with no name.
        @Override
        public void move(){
            System.out.println("Move up (Y+1)");
        }
    },

    SOUTH{
        @Override
        public void move(){
            System.out.println("Move down (Y-1)");
        }
    },

    EAST{
        @Override
        public void move(){
            System.out.println("Move right (x+1)");
        }
    },

    WEST{
        @Override
        public void move(){
            System.out.println("Move left (x-1)");
        }
    };

    public abstract void move(); //Abstract Method, or isme hume class ko abstract krne ki zarurat
    nahi ha.
}
```

```
enum Direction {
    NORTH {
        @Override
        public void move() {
            System.out.println("Moving Up");
        }
    },
    SOUTH {
        @Override
        public void move() {
            System.out.println("Moving Down");
        }
    },
    EAST {
        @Override
        public void move() {
            System.out.println("Moving Right");
        }
    },
    WEST {
        @Override
        public void move() {
            System.out.println("Moving Left");
        }
    };
    public abstract void move();
}
```

```
final class Direction extends Enum<Direction> {
    public static final Direction NORTH = new Direction() {
        @Override
        public void move() {

        }
    };
    public static final Direction EAST = new Direction() {
        @Override
        public void move() {

        }
    };
    public static final Direction SOUTH = new Direction(){
        @Override
        public void move() {

        }
    };
    public static final Direction WEST = new Direction(){
        @Override
        public void move() {

        }
    };

    public abstract void move();
}
```

class Enum → default methods in “class Enum”.

- **values()**: myth that this comes from class Enum but this method is compile generated.
- **valueOf(String)**: this is also a myth that it comes from class Enum and this is also a compile generated method.
- **name()**: Yes, this is a method that comes from enum class, it is used to get name of the enum Direction → constants values → EAST, WEST, NORTH, SOUTH., and this method can't be override, because it is final.
- **ordinal()**: Yes, this is also a method that comes from enum class.

```
public class MethodsOfEnumClass {
    public static void main(String[] args) {

        //IF i call values();--> this is out of enum class, it is compiler generated method.
        // this is used taki hum "enum Direction" ki sari values laa sake.
        Direction[] directions = Direction.values();
        for(Direction d : directions){
            System.out.println(d.name());
        }

        //valueOf();--> this is also out of enum class, it is also compile generated.
        //ye sirf string ko constant me convert krta ha, or ye case sensitive hota ha.
        Direction d1 = Direction.EAST;
        Direction d2 = Direction.valueOf("EAST"); // CASE SENSITIVE.
        System.out.println(d2.name());

        // name();--> it is a method from enum class, this is final can't be @Override.
        System.out.println(d1.name());
        System.out.println(d2.toString()); // both are same but this can be @Override.
        // ordinal();--> this gives the value of constants.
        System.out.println(d1.ordinal()); // ans--> 2 because "EAST" is in 2nd place.

    }
}

enum Direction{
    NORTH,
    SOUTH,
    EAST,
    WEST;
}
```